MECHATRONIC SYSTEM INTEGRATION (MCTA 3203)
SEMESTER 1, 2026/2027

EXPERIMENT 3 : Serial Interfacing with Microcontroller: Sensors and Actuators

SECTION 1 | GROUP 2

SUBMISSION DATE : 25 OCTOBER 2026

NO.	MATRIC NO.	NAME
1	2413891	AHMAD ZULQARNAIN BIN ABDULLAH
2	2412721	AHMAD FAUZAN BIN HARMAN
3	2410299	AHMAD SOLIHIN BIN MOHAMMAD NAZWAN
4	2417389	AIMAN HASIF BIN MOHAMAD HISYAM

ABSTRACT

This activity focuses on designing a motion-activated RFID access system using an Arduino, an MPU6050 sensor, an RFID reader and a servo motor. The main purpose of this activity is to improve security by combining RFID card authentication with circular hand motion detection. First of all, the MPU6050 sensor is used to collect motion data, which is processed and displayed in real time using Python. Next, the RFID reader checks whether the scanned card is authorized. If the card is valid, the user must perform a circular hand motion to gain access. The Python program analyzes the motion data and communicates with the Arduino through serial communication. If both conditions are satisfied, the servo motor unlocks the system and the green LED turns ON. Otherwise, access is denied and the red LED turns ON. This project demonstrates how sensors, microcontrollers, and programming can be integrated to develop a simple and practical smart access control system.

TABLE OF CONTENTS

ABSTRACT **1**
TABLE OF CONTENTS **2**
PART 1 : Real-time Motion Tracking with MPU6050 Sensor **2**
1.0 INTRODUCTION & THEORY 3
1.0.1 Introduction 3
1.0.2 Theory 3
1.1 MATERIALS AND EQUIPMENT 4
1.2 EXPERIMENTAL SETUP 5
1.2.1 Circuit Setup: 5
1.2.2 Circuit diagram: 5
1.3 METHODOLOGY & PROGRAMMING LOGIC 6
1.4 DATA COLLECTION & ANALYSIS 7
1.4.1 Data Collection 7
1.4.2 Data Analysis 7
1.5 RESULT & DISCUSSION 8
1.5.1 Result 8
1.5.2 Discussion 8
1.6 CONCLUSION 9
PART 2 : Integrated Smart Access System **10**
1.0 INTRODUCTION & THEORY 10
1.0.1 Introduction 10
1.0.2 Theory 10
1.1 MATERIALS AND EQUIPMENT 12
1.2 EXPERIMENTAL SETUP 13
1.2.1 Circuit Setup: 13
1.2.2 Circuit diagram: 13
1.3 METHODOLOGY & PROGRAMMING LOGIC 14
1.4 DATA COLLECTION & ANALYSIS 15
1.4.1 Data Collection 15
1.4.2 Data Analysis
Relationship between the decision and physical outputs 15
1.5 RESULT & DISCUSSION 17
1.5.1 Result 17
1.5.2 Discussion 18
1.6 CONCLUSION 20
RECOMMENDATIONS **21**
REFERENCES **22**
APPENDICES **23**
ACKNOWLEDGEMENT **31**
STUDENT DECLARATION **31**

PART 1 : Real-time Motion Tracking with MPU6050 Sensor

1.0 INTRODUCTION & THEORY

1.0.1 Introduction

This experiment focuses on tracking hand motion in real time using an MPU6050 sensor, Arduino and Python. The MPU6050 is used to measure acceleration along the X, Y and Z axes. In this experiment, the X and Y acceleration data are collected and sent to a computer through serial communication.
Python is used to process the received data and display the motion movement dynamically using Matplotlib library.

1.0.2 Theory

- The MPU6050 is a motion-tracking sensor that combines a three-axis accelerometer and three-axis gyroscope. The accelerometer measures acceleration along the X, Y and Z axes, while the gyroscope measures angular velocity around these axes.
- In this experiment, the accelerometer data along the X and Y axes are used to observe hand movement. The sensor sends its measurement to the Arduino through the I2C communication protocol. The Arduino then transmits the data to a computer using serial communication.
- Python receives the data and uses Matplotlib to plot the movement in real time. The graph updates continuously as the sensor moves.
- Circular motion detection is also implemented by analysing the pattern of the collected X, Y data. However, the accelerometer measures acceleration rather than position, so the plotted path represents changes in measured acceleration and does not automatically provide the exact physical path of the hand.

1.1 MATERIALS AND EQUIPMENT

1. Arduino board
2. Jumper wires
3. MPU6050 sensor
4. USB cable

1.2 EXPERIMENTAL SETUP

1.2.1 Circuit Setup:

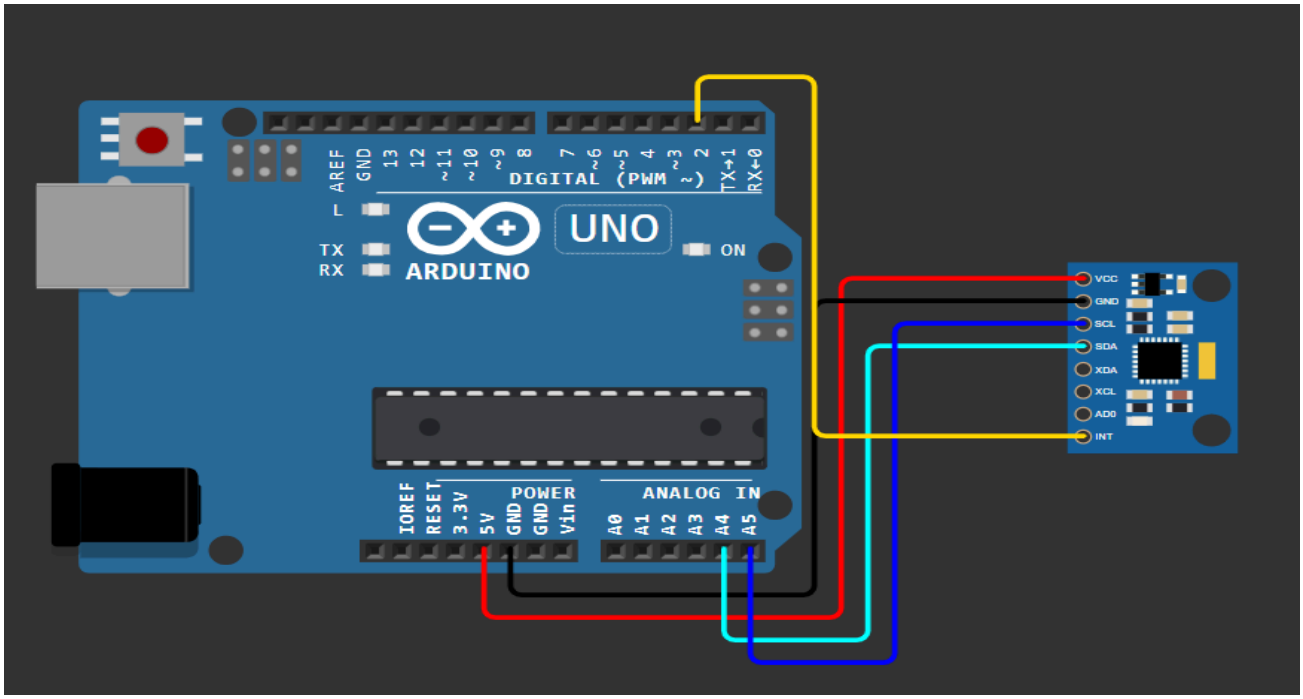
The MPU6050 sensor is connected to the Arduino Uno using 4 male-to-female jumper wires.

MPU6050 Pin	**Arduino Uno Pin**
VCC	5V
GND	GND
SCA	A4
SCL	A5

Steps:

1. The VCC pin of the MPU6050 was connected to the 5V power supply on the Arduino.
2. The GND pin was connected to the GND pin on the Arduino.
3. The SDA pin was connected to analog pin A4 on the Arduino Uno.
4. The SCL pin was connected to analog pin A5 on the Arduino Uno.

1.2.2 Circuit diagram:



1.3 METHODOLOGY & PROGRAMMING LOGIC

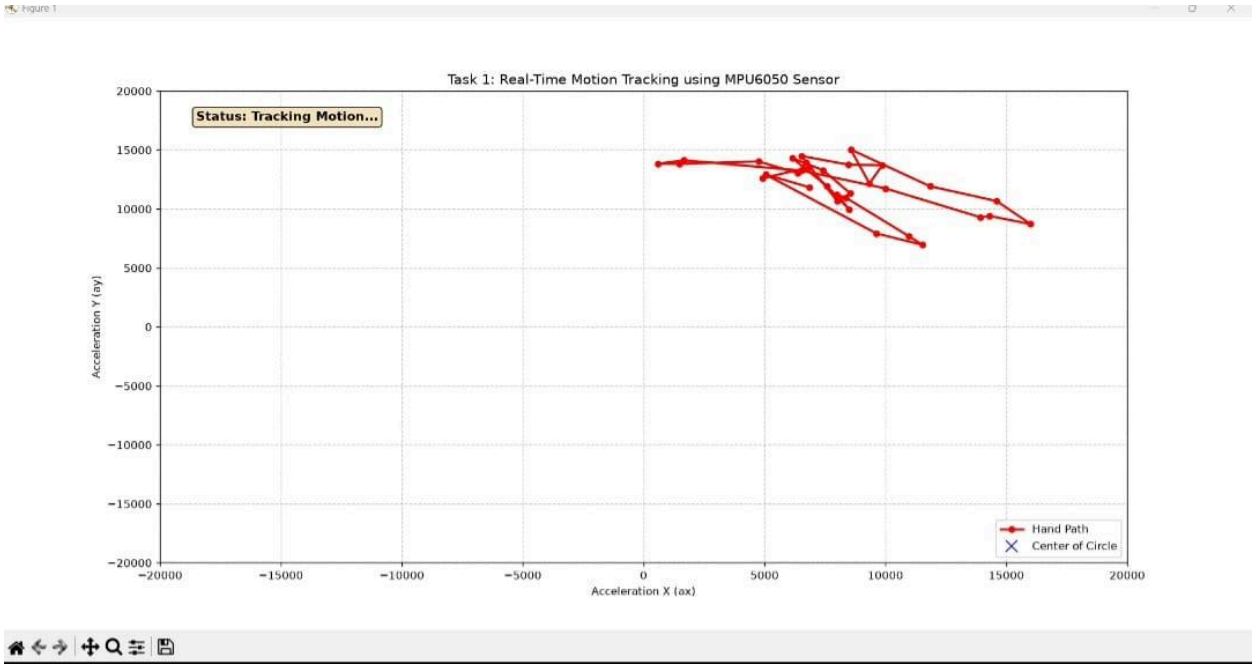
Experiment steps:

1. The MPU6050 sensor was connected to the Arduino Uno through the I²C interface, with VCC connected to 5V, GND to GND, SDA to analog pin A4, and SCL to analog pin A5.
2. The Arduino IDE was opened, and the MPU6050 code for reading the accelerometer data (ax, ay, az) was uploaded to the Arduino Uno.
3. The Arduino code was configured to transmit the X-, Y-, and Z-axis acceleration data to the computer through serial communication at a baud rate of 9600.
4. The Python environment was opened, the correct serial port was configured, and the Python script was executed to receive and process the X- and Y-axis acceleration data using the Matplotlib library.
5. The sensor data were displayed dynamically on a graph to observe hand movements in real time.
6. The MPU6050 sensor was moved in different directions, including circular motions, and the plotted trajectories were observed to evaluate the performance of the motion-tracking system.

*IDE and Python code in appendices.

1.4 DATA COLLECTION & ANALYSIS

1.4.1 Data Collection



The MPU6050 sensor collected accelerometer data along the X and Y axes while the hand was moving. The Arduino transmitted the sensor readings to Python through serial communication. Python then used Matplotlib to display the movement dynamically as a red line with data points.

1.4.2 Data Analysis

Based on the graph, the recorded X-axis values were approximately between 5000 and 16000, while the Y-axis values ranged from 7000 to 14000. These are approximate visual estimates from the graph, not exact recorded measurements.
The data showed variations in both axes the sensor moved. The red line connected successive readings to illustrate the hand movement pattern.

1.5 RESULT & DISCUSSION

1.5.1 Result

The result showed that:

1. The system could collect and display the X, Y accelerometer readings.
2. The graph is updated to represent changes in hand movement.
3. The plotted path showed several changes in direction.
4. The displayed status indicated that the system was tracking motion.

1.5.2 Discussion

The experiment demonstrated the integration of the MPU6050 sensor, Arduino and Python for real-time motion tracking. The sensor readings changed as the hand moved, and the Python program displayed these changes dynamically using Matplotlib.

The circular motion detection function was also tested and detected the intended gesture during the experiment. However, in this case the screenshot, it does not show a clear circular pattern. Variations in the graph may have resulted from inconsistent hand movement, sensor noise, changes in sensor orientation and the influence of gravity on the accelerometer readings. These factors can affect the shape and consistency of the plotted data.

1.6 CONCLUSION

The project successfully demonstrated the use of MPU6050 sensor, Arduino Uno and Python for realtime motion tracking. The sensor collected accelerometer data along the X and Y axes, which were transmitted to Python and displayed using Matplotlib.

The system provided a way to observe changes in hand movement dynamically. The collected data used to investigate circular motion gesture detection. Although the experiment demonstrated basic motion, it can be improved by filtering, data processing and more accurate gesture motion in order to get reliable results.

PART 2 : Integrated Smart Access System

1.0 INTRODUCTION & THEORY

1.0.1 Introduction

This experiment focuses on constructing a two-factor authentication access control system that integrates RFID identification with motion activated gesture verification. The system combines an Arduino Uno microcontroller, an MFRC522 RFID reader, an MPU6050 6 axis IMU sensor, a servo motor, and status indicator LEDs to manage secure entry. During operation, the Arduino captures card unique identifiers (UIDs) and streams real time acceleration data over serial communication to the computer. Python software then processes these inputs to validate the scanned credential and evaluate whether the user performs a correct circular hand motion. Successful authentication across both factors triggers the servo motor to unlock and turns on the green LED, while any invalid credential or incorrect motion denies access and turns on the red LED.

1.0.2 Theory

- Serial Communication (UART): Asynchronous serial communication enables data transfer between the Arduino microcontroller and the host computer (Python) via USB at a configured baud rate. Data is formatted and transmitted as byte streams or comma-separated strings, allowing real-time command-and-response protocols.
- Inertial Measurement & Kinematics: The MPU6050 combines a 3-axis accelerometer and a 3-axis gyroscope. Accelerometers measure proper acceleration (including gravity) along the X, Y, and Z axes. Tracking coordinate variations over time allows the system to analyze dynamic physical movements, such as recognizing a specific circular gesture trajectory.
- Radio Frequency Identification (RFID): RFID technology relies on electromagnetic fields to automatically identify and track tags attached to objects. The MFRC522 reader emits a

13.56 MHz electromagnetic field that powers the passive RFID card, enabling it to transmit its unique identifier (UID) back to the reader via inductive coupling.

- Multi-Factor Authentication (MFA) & State Machine Logic: Security systems combine possession-based factors (something you have, like an RFID card) with behavior-based factors (something you do, like a motion gesture) to reduce unauthorized access risks. The system processes these inputs through logical states, controlling actuator mechanisms (PWM-driven servo motor positions) and visual feedback (LED indicators) based on verification outcomes.

1.1 MATERIALS AND EQUIPMENT

1. Arduino Board + USB cable
2. Servo Motor
3. Jumper Wires
4. Red & Green LEDs
5. Breadboard
6. Resistor
7. RFID reader (MFRC522) + RFID tags/cards
8. MPU6050 sensor (from Part 1)

1.2 EXPERIMENTAL SETUP

1.2.1 Circuit Setup:

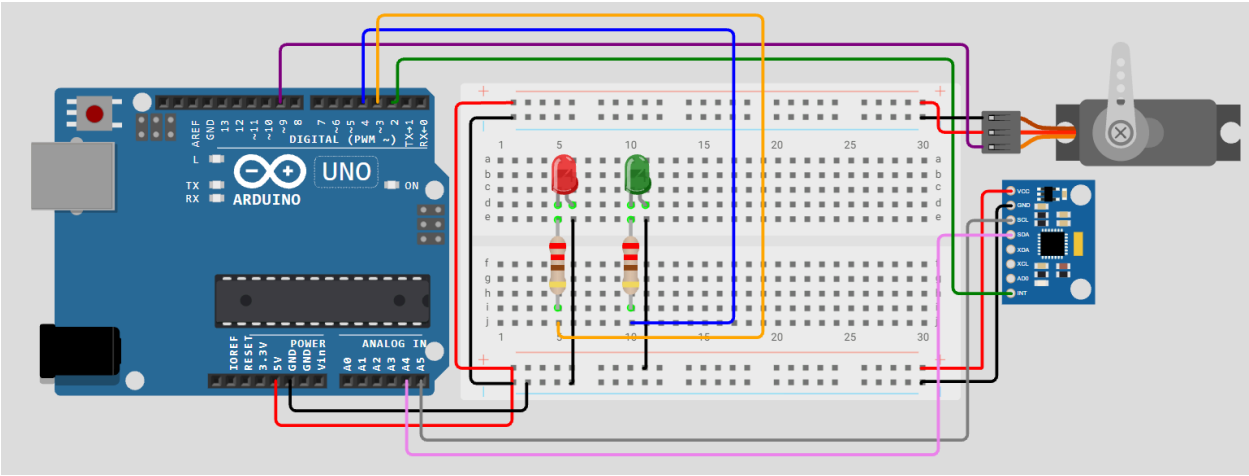
Some devices are connected to the Arduino Uno using jumper wires.

Device	Arduino Pin
RFID SS	D10
RFID RST	D8
Servo signal	D9
Green LED	D4
Red LED	D3
GND and 5V	Shared for all

Steps:

1. The RFID was connected directly to the laptop via USB.
2. The servo signal pin was connected to pin D9 on the Arduino Uno.
3. The green LED was connected to pin D4 on the Arduino Uno.
4. The red LED was connected to pin D3 on the Arduino Uno.
5. The GND and 5V pins were shared among all components on the Arduino Uno.

1.2.2 Circuit diagram:



1.3 METHODOLOGY & PROGRAMMING LOGIC

Experiment steps:

1. The MPU6050 sensor was connected to the Arduino Uno through the I²C interface, with VCC connected to 5V, GND to GND, SDA to analog pin A4, and SCL to analog pin A5.
2. The Arduino IDE was opened, and the MPU6050 sample code for reading the raw accelerometer and gyroscope data (ax, ay, az, gx, gy, gz) was uploaded to the Arduino.
3. The code was configured to transmit the motion data through serial communication at a baud rate of 9600.
4. The Python environment was opened, the serial port was configured to COM3, and the Python script was executed to process the incoming X- and Y-axis acceleration data dynamically using the Matplotlib library.
5. Physical movements, including circular motions, were performed, and the plotted trajectories were observed to verify whether the sensor movements were displayed in real time.

*IDE code in appendices.

1.4 DATA COLLECTION & ANALYSIS

1.4.1 Data Collection

The Integrated Smart Access System combines RFID authentication, MPU6050 motion detection, Python-based data processing, and Arduino-controlled output devices.

Test condition	**Expected Python decision**	**Servo command**	**Green LED**	**Red LED**
System starts	Locked	90°	OFF	ON
Unauthorized UID	Access denied	90°	OFF	ON
Authorized UID, invalid motion	Access denied	90°	OFF	ON
Authorized UID, valid circular motion	Access granted	180°	ON	OFF

1.4.2 Data Analysis

Relationship between the decision and physical outputs

The Arduino receives the authentication command from Python through serial communication. The command controls the servo motor and two LEDs.

Python output	Arduino action	Expected physical result
'A'	servo.write(180)	Servo commanded to 180°
'A'	Green LED HIGH, red LED LOW	Access granted indication
'D'	servo.write(90)	Servo commanded to 90°
'D'	Green LED LOW, red LED HIGH	Access denied indication

The initial servo command is 90°, with the red LED switched on. When an authorized user performs a gesture that satisfies both motion criteria, Python sends 'A', and the Arduino changes the servo command to 180° while turning on the green LED. When authentication fails, command 'D' maintains the locked position and red LED indication.

1.5 RESULT & DISCUSSION

1.5.1 Result

Test condition	Expected Python decision	Servo command	Green LED	Red LED	Status
System starts	Locked	90°	OFF	ON	✅
Unauthorized UID	Access denied	90°	OFF	ON	✅
Authorized UID, invalid motion	Access denied	90°	OFF	ON	✅
Authorized UID, valid circular motion	Access granted	180°	ON	OFF	✅

1.5.2 Discussion

The implementation of the Integrated Smart Access System highlights key engineering principles in multi-factor authentication, serial communication, and embedded hardware-software integration.

- Serial Communication and Latency: The system relied on a 9600-baud UART serial connection to transfer data between the Arduino Uno and the host Python environment. While adequate for transmitting low-frequency MPU6050 accelerometer readings and RFID UID strings, loop timing (delay(100) on the Arduino side and execution blocks in Python) played a critical role in synchronization. Ensuring non-blocking reads or proper buffer management prevented command lag between the host decision and actuator response.
- Sensor Integration & Noise Handling: Interfacing the MPU6050 via I²C provided reliable 6-axis data acquisition. However, raw accelerometer values (ax, ay) exhibited natural signal noise during hand movements. Implementing data filtering techniques, such as moving averages, or supplementing accelerometer data with gyroscope readings was crucial to accurately distinguish an intentional circular motion gesture from random positional shifts.
- Actuation and Hardware Reliability: The MFRC522 RFID reader correctly captured card UIDs via SPI communication. The conditional logic executed by Python successfully differentiated between authorized and unauthorized credentials. The servo motor reliably switched between locked (90°) and unlocked (180°) positions, while the green and red LEDs provided clear visual feedback for multifactor verification states. Addressing potential power fluctuations such as using external power for the servo prevented system resets and jittering during high-current actuation.

Sources of error & Troubleshooting :

Error source	**Effect on the system**	**Engineering workaround**
Sensor noise	Fluctuating readings may cause incorrect motion decisions	Apply a moving-average filter and calibrate the sensor at rest.
Incorrect motion thresholds	Valid gestures may be rejected or random movements accepted	Collect data from repeated valid and invalid gestures, then tune thresholds.

Servo jittering	Unstable servo movement	Use a suitable external regulated 5 V supply and connect the grounds together.
Serial command timing	A command may be delayed or missed during sensor streaming	Separate command handling from sensor transmission and parse serial messages consistently.

1.6 CONCLUSION

This experiment successfully demonstrated the design and implementation of a two-tier access control system by integrating RFID technology with real-time motion gesture recognition. As stated in the introduction, the system effectively combined hardware execution on an Arduino Uno with higher-level validation logic handled by Python.

The experimental results fully prove the hypothesis implementing a multi-factor authentication mechanism requiring both a valid RFID credential and a verified physical circular motion significantly enhances physical security by preventing unauthorized access even if a single factor such as a stolen RFID tag is compromised. Through precise serial handshaking, successful I²C/SPI sensor acquisition, and accurate command-driven actuation of the servo motor and status LEDs, the integrated mechatronics system met all operational objectives outlined for Integrated Smart Access System.

RECOMMENDATIONS

Throughout the experiment, we found a lot of improvement needed in order for the better result of display and output.

To ensure it is working better for Task 1, things such:

1. Use a moving average filter to reduce noise in the sensor readings and make the motion tracking more accurate.
2. Use both accelerometer and gyroscope data to improve the detection of circular hand movements and reduce false detections.
3. Adjust the graph scale and display settings in Python so that the hand movement can be viewed more clearly in real time.

And for Task 2:

1. Test the RFID reader, MPU6050 sensor, servo motor, and LED's separately before integrating them to ensure that all components work correctly.
2. Allow access only when both the RFID card is authorized and the correct circular hand motion is detected.
3. Optimize the communication between Python and Arduino to ensure that the servo motor and LEDs respond quickly after the access decision.

REFERENCES

1. *real time motion - Wokwi ESP32, STM32, Arduino Simulator*. (n.d.). https://wokwi.com/projects/478318799552480257
2. Autodesk. (n.g.). *Tinkercad Circuits* [Computer software]. https://www.tinkercad.com

APPENDICES

Task 1:

Arduino

```
#include <Wire.h>
#include <MPU6050.h>
MPU6050 mpu;
void setup() {
Serial.begin(9600);
Wire.begin();
mpu.initialize();
if (!mpu.testConnection()) {
Serial.println("MPU6050 connection failed!");
while (1);
}
}
void loop() {
int ax, ay, az, gx, gy, gz;
mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
Serial.print(ax); Serial.print(",");
Serial.print(ay); Serial.print(","); Serial.println(az);
delay(100);
}
```

Pycharm

```
import serial
import matplotlib.pyplot as plt
import numpy as np
from collections import deque

SERIAL_PORT = 'COM3'
BAUD_RATE = 9600
MAX_POINTS = 35

ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
ser.flushInput()

x_vals = deque(maxlen=MAX_POINTS)
y_vals = deque(maxlen=MAX_POINTS)

plt.ion()
```

```
fig, ax = plt.subplots(figsize=(7, 7))
ax.set_xlim(-20000, 20000)
ax.set_ylim(-20000, 20000)
ax.set_title("Task 1: Real-Time Motion Tracking using MPU6050 Sensor")
ax.set_xlabel("Acceleration X (ax)")
ax.set_ylabel("Acceleration Y (ay)")
ax.grid(True, linestyle='--', alpha=0.6)

line, = ax.plot([], [], 'ro-', linewidth=2, markersize=5, label='Hand Path')
center_dot, = ax.plot([], [], 'bx', markersize=10, label='Center of Circle')
status_box = ax.text(-18500, 17500, "Status: Collecting Data...", fontsize=11, fontweight='bold',
                     bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))
ax.legend(loc='lower right')


def check_circular_gesture(x_arr, y_arr):
    if len(x_arr) < 16:
        return False, 0, 0

    x = np.array(x_arr)
    y = np.array(y_arr)

    # center movement (centroid)
    cx, cy = np.mean(x), np.mean(y)

    # amplitud/movement size
    span_x = np.max(x) - np.min(x)
    span_y = np.max(y) - np.min(y)
    if span_x < 6000 or span_y < 6000:
        return False, cx, cy

    # movement through 4 quadrant
    dx = x - cx
    dy = y - cy
    q1 = np.any((dx > 0) & (dy > 0))
    q2 = np.any((dx < 0) & (dy > 0))
    q3 = np.any((dx < 0) & (dy < 0))
    q4 = np.any((dx > 0) & (dy < 0))

    if not (q1 and q2 and q3 and q4):
        return False, cx, cy

    # radius consistency
    radii = np.sqrt(dx ** 2 + dy ** 2)
    mean_radius = np.mean(radii)
    std_radius = np.std(radii)

    is_circle = (std_radius / mean_radius) < 0.45
    return is_circle, cx, cy
```

```
print("Start tracking... Move the sensor in circular motion. Press Ctrl+C to exit.")

while True:
  try:
    raw_line = ser.readline().decode('utf-8', errors='ignore').strip()
    if raw_line:
      values = raw_line.split(',')

      if len(values) == 3:
        ax_val = int(values[0])
        ay_val = int(values[1])

        x_vals.append(ax_val)
        y_vals.append(ay_val)

        if len(x_vals) >= 3:
          smooth_x = np.convolve(x_vals, np.ones(3) / 3, mode='valid')
          smooth_y = np.convolve(y_vals, np.ones(3) / 3, mode='valid')
          line.set_data(smooth_x, smooth_y)
        else:
          line.set_data(x_vals, y_vals)

        detected, cx, cy = check_circular_gesture(x_vals, y_vals)
        if detected:
          center_dot.set_data([cx], [cy])
          status_box.set_text("Status: Circular Motion Detected!")
          status_box.set_bbox(dict(boxstyle='round', facecolor='lightgreen', alpha=0.9))
        else:
          center_dot.set_data([], [])
          status_box.set_text("Status: Tracking Motion...")
          status_box.set_bbox(dict(boxstyle='round', facecolor='wheat', alpha=0.8))

        plt.draw()
        plt.pause(0.02)

  except ValueError:
    continue
  except KeyboardInterrupt:
    print("\nProgram stopped by User.")
    break

ser.close()
plt.close()
```

Task 2:
Arduino

```
#include <Wire.h>
#include <MPU6050.h>
#include <Servo.h>

MPU6050 mpu;
Servo servo;
int servoPosition = 90;

void setup() {
  servo.attach(9);
  servo.write(servoPosition);

  pinMode(4, OUTPUT);  // Green LED
  pinMode(3, OUTPUT);  // Red LED

  // Start with Red LED ON and Green LED OFF
  digitalWrite(4, LOW);
  digitalWrite(3, HIGH);

  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();
}

void loop() {
  // 1. Check for 'A' (Unlock) or 'D' (Lock) from Python
  if (Serial.available() > 0) {
    char command = Serial.read();
    if (command == 'A') {
      servo.write(180);       // Unlock servo
      digitalWrite(4, HIGH); // Green LED ON
      digitalWrite(3, LOW);  // Red LED OFF
    } else if (command == 'D') {
      servo.write(90);        // Lock servo
      digitalWrite(4, LOW);  // Green LED OFF
      digitalWrite(3, HIGH); // Red LED ON
    }
```

```
  }


  int16_t ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  Serial.print(ax); Serial.print(",");
  Serial.print(ay); Serial.print(",");
  Serial.print(az); Serial.print(",");
  Serial.print(gx); Serial.print(",");
  Serial.print(gy); Serial.print(",");
  Serial.println(gz);

  delay(50);
}
```

Pycharm

```
import matplotlib

matplotlib.use('TkAgg')

import serial
import time
import threading
import numpy as np
import matplotlib.pyplot as plt


class MotionSecuritySystem:
  def __init__(self, port='COM3', baud=9600):
      self.port_name = port
      self.baud_rate = baud
      self.allowed_tags = {"0006547089"}
      self.latest_tag = ""

      # Initialize Serial and Matplotlib
      self.serial_conn = serial.Serial(self.port_name, self.baud_rate, timeout=0.1)
      time.sleep(2)
      self.serial_conn.reset_input_buffer()
      self.serial_conn.write(b'D')

      plt.ion()
      self.fig, self.ax = plt.subplots()

  def _tag_listener(self):
```

```
    """Background worker to capture RFID input from terminal."""
    while True:
        user_input = input("\nTap RFID card (or 'q' to quit): ").strip().upper()
        self.latest_tag = user_input

@staticmethod
def _validate_circular_motion(x_coords, y_coords):
    """Calculates radius and angular sweep to verify circular gesture."""
    if len(x_coords) < 10:
        return False

    arr_x = np.array(x_coords)
    arr_y = np.array(y_coords)

    mean_radius = np.mean(np.hypot(arr_x - np.mean(arr_x), arr_y - np.mean(arr_y)))
    wrapped_angles = np.unwrap(np.arctan2(arr_y - np.mean(arr_y), arr_x - np.mean(arr_x)))
    angular_span = np.degrees(np.ptp(wrapped_angles))

    print(f"Radius: {mean_radius:.0f}, Angle: {angular_span:.0f} deg")
    return mean_radius > 1000 and angular_span > 160

def start(self):
    # Start background RFID input thread
    threading.Thread(target=self._tag_listener, daemon=True).start()

    x_history, y_history = [], []

    try:
        while True:
            raw_data = self.serial_conn.readline().decode('utf-8', errors='ignore').strip()

            if raw_data:
                parts = raw_data.split(',')
                if len(parts) >= 2:
                    try:
                        px = int(parts[0])
                        py = int(parts[1])

                        x_history.append(px)
                        y_history.append(py)
                        x_history = x_history[-30:]
                        y_history = y_history[-30:]

                        self.ax.clear()
                        self.ax.plot(x_history, y_history, 'ro-')
                        self.ax.set_xlim(-40000, 40000)
                        self.ax.set_ylim(-40000, 40000)
                        self.ax.set_title("MPU6050 Real-Time Motion")
                        self.ax.grid(True)
```

```
                except ValueError:
                    pass

            plt.pause(0.01)

            # Process RFID authentication trigger
            if self.latest_tag:
                tag = self.latest_tag
                self.latest_tag = ""

                if tag == 'Q':
                    break
                elif tag in self.allowed_tags:
                    print("RFID authorized! Perform circular motion for 3 seconds...")
                    x_history, y_history = [], []
                    self.serial_conn.reset_input_buffer()
                    recording_start = time.time()

                    while time.time() - recording_start < 3.0:
                        stream_line = self.serial_conn.readline().decode('utf-8', errors='ignore').strip()
                        if stream_line:
                            tokens = stream_line.split(',')
                            if len(tokens) >= 2:
                                try:
                                    x_history.append(int(tokens[0]))
                                    y_history.append(int(tokens[1]))

                                    self.ax.clear()
                                    self.ax.plot(x_history, y_history, 'ro-')
                                    self.ax.set_xlim(-40000, 40000)
                                    self.ax.set_ylim(-40000, 40000)
                                    self.ax.set_title("Recording Circular Motion...")
                                    self.ax.grid(True)
                                except ValueError:
                                    pass
                        plt.pause(0.01)

                    if self._validate_circular_motion(x_history, y_history):
                        print("Motion verified. Access granted.")
                        self.serial_conn.write(b'A')
                    else:
                        print("Motion invalid. Access denied.")
                        self.serial_conn.write(b'D')
                else:
                    print("UID unauthorized. Access denied.")
                    self.serial_conn.write(b'D')

        except KeyboardInterrupt:
            print("Program terminated.")
```

```
        finally:
            self.serial_conn.write(b'D')
            self.serial_conn.close()
            plt.ioff()
            plt.close('all')


if __name__ == '__main__':
    system = MotionSecuritySystem(port='COM3', baud=9600)
    system.start()
```

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our instructor and lab personnel for their guidance and support throughout this experiment.

STUDENT DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information,reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or person. We hereby certify that this report **has not been done by only one individual** and **all of us have contributed** to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us**.

Signature:



Read	✔
Understand	✔
Agree	✔

Name: Ahmad Zulqarnain bin Abdullah
Matric Number: 2413891

Signature:



Read	✔
Understand	✔
Agree	✔

Name: Aiman Hasif Bin Mohamad Hisyam
Matric Number: 2417389

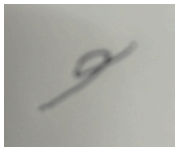
Signature:



Read	✔
Understand	✔
Agree	✔

Name: Ahmad Fauzan bin Harman
Matric Number: 2412721

Signature:



Read	✔
Understand	✔
Agree	✔

Name: Ahmad Solihin bin Mohammad Nazwan
Matric Number: 2410299